

LAB 7 – Joins:

Inner Join: -

```
CREATE TABLE Department (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(30)  
);
```

```
CREATE TABLE Employee (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(30),  
    dept_id INT  
);
```

```
CREATE TABLE Staff (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(30),  
    manager_id INT  
);
```

```
INSERT INTO Department VALUES (10, 'Sales'), (20, 'HR'), (30, 'IT'), (40, 'Finance');
```

```
INSERT INTO Employee VALUES  
(1, 'Asha', 10),  
(2, 'Ravi', 20),  
(3, 'Meena', 30),  
(4, 'John', NULL);
```

```
INSERT INTO Staff VALUES  
(1, 'Asha', NULL),  
(2, 'Ravi', 1),  
(3, 'Meena', 1),  
(4, 'John', 2);
```

```
SELECT e.emp_id, e.emp_name, d.dept_name  
FROM Employee e  
INNER JOIN Department d ON e.dept_id = d.dept_id;
```

	emp_id	emp_name	dept_name
▶	1	Asha	Sales
	2	Ravi	HR
	3	Meena	IT

Left Join: -

```
38 • SELECT e.emp_id, e.emp_name, d.dept_name
39 FROM Employee e
40 LEFT JOIN Department d ON e.dept_id = d.dept_id;
```

	emp_id	emp_name	dept_name
▶	1	Asha	Sales
	2	Ravi	HR
	3	Meena	IT
	4	John	NULL

Right Join: -

```
38 • SELECT e.emp_id, e.emp_name, d.dept_name
39 FROM Employee e
40 RIGHT JOIN Department d ON e.dept_id = d.dept_id;
```

	emp_id	emp_name	dept_name
▶	1	Asha	Sales
	2	Ravi	HR
	3	Meena	IT
	NULL	NULL	Finance

Fully Outer: -

```

38 • SELECT e.emp_id, e.emp_name, d.dept_name
39 FROM Employee e
40 LEFT JOIN Department d ON e.dept_id = d.dept_id
41 UNION
42 SELECT e.emp_id, e.emp_name, d.dept_name
43 FROM Employee e
44 RIGHT JOIN Department d ON e.dept_id = d.dept_id;

```

Result Grid   Filter Rows: Export:  Wrap Cell Content: 

	emp_id	emp_name	dept_name
▶	1	Asha	Sales
	2	Ravi	HR
	3	Meena	IT
	4	John	NULL
	NULL	NULL	Finance

Cross Join: -

```

SELECT e.emp_name, d.dept_name
FROM Employee e
CROSS JOIN Department d;

```

	emp_name	dept_name
▶	John	Sales
	Meena	Sales
	Ravi	Sales
	Asha	Sales
	John	HR
	Meena	HR
	Ravi	HR
	Asha	HR
	John	IT
	Meena	IT
	Ravi	IT
	Asha	IT
	John	Finance
	Meena	Finance
	Ravi	Finance
	Asha	Finance

Self-Join: -

```
38 • SELECT e.emp_name AS employee, m.emp_name AS manager
39 FROM Staff e
40 INNER JOIN Staff m ON e.manager_id = m.emp_id;
```

	employee	manager
▶	Ravi	Asha
	Meena	Asha
	John	Ravi

LAB - 8: -

Group by, Having and Order by Clause: -

Group by -

```
6 • CREATE TABLE Emp (
7     emp_id INT PRIMARY KEY,
8     emp_name VARCHAR(30),
9     dept VARCHAR(30),
10    salary INT
11 );
12
13 • INSERT INTO Emp VALUES
14 (1, 'Asha', 'Sales', 50000),
15 (2, 'Ravi', 'HR', 40000),
16 (3, 'Meena', 'IT', 60000),
17 (4, 'John', 'IT', 55000),
18 (5, 'Priya', 'Sales', 45000),
19 (6, 'Kiran', 'HR', 42000),
20 (7, 'Sneha', 'IT', 65000);
21
```

```

22 • SELECT dept,
23       COUNT(*) AS total_emp,
24       SUM(salary) AS total_salary,
25       AVG(salary) AS avg_salary
26 FROM Emp
27 GROUP BY dept;

```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	dept	total_emp	total_salary	avg_salary
▶	Sales	2	95000	47500.0000
	HR	2	82000	41000.0000
	IT	3	180000	60000.0000

Having by -

```

22 • SELECT dept, COUNT(*) AS total_emp
23 FROM Emp
24 GROUP BY dept
25 HAVING COUNT(*) > 2;

```

Result Grid   Filter Rows: | Export:  | Wrap Cell Content: 

	dept	total_emp
▶	IT	3

Order by -

```

22 • SELECT emp_name, dept, salary
23 FROM Emp
24 ORDER BY salary DESC;

```

Result Grid			Filter Rows:
	emp_name	dept	salary
▶	Sneha	IT	65000
	Meena	IT	60000
	John	IT	55000
	Asha	Sales	50000
	Priya	Sales	45000
	Kiran	HR	42000
	Ravi	HR	40000

Group by + having + order by

```
22 • SELECT dept, AVG(salary) AS avg_salary
23 FROM Emp
24 GROUP BY dept
25 HAVING AVG(salary) > 45000
26 ORDER BY avg_salary DESC;
```

Result Grid		Filter Rows:	Export:
	dept	avg_salary	
▶	IT	60000.0000	
	Sales	47500.0000	

LAB – 9: -

Procedure: -

Basic Procedure:

```
CREATE TABLE Emp (
    emp_id    INT PRIMARY KEY,
    emp_name  VARCHAR(30),
    dept      VARCHAR(30),
    salary    INT
);
```

```
INSERT INTO Emp VALUES
(1, 'Asha', 'Sales', 50000),
(2, 'Ravi', 'HR', 40000),
(3, 'Meena', 'IT', 60000),
(4, 'John', 'IT', 55000),
(5, 'Priya', 'Sales', 45000),
(6, 'Kiran', 'HR', 42000),
(7, 'Sneha', 'IT', 65000);
```

```
DROP PROCEDURE IF EXISTS show_emp;
```

```
DELIMITER $$
```

```
CREATE PROCEDURE show_emp()
```

```
BEGIN
```

```
    SELECT * FROM Emp;
```

```
END$$
```

```
DELIMITER ;
```

```
CALL show_emp();
```

	emp_id	emp_name	dept	salary
▶	1	Asha	Sales	50000
	2	Ravi	HR	40000
	3	Meena	IT	60000
	4	John	IT	55000
	5	Priya	Sales	45000
	6	Kiran	HR	42000
	7	Sneha	IT	65000

Procedure for Insert:

```
DELIMITER $$
```

```
CREATE PROCEDURE insert_emp(
```

```
    IN p_id    INT,
```

```
    IN p_name  VARCHAR(30),
```

```
    IN p_dept  VARCHAR(30),
```

```
    IN p_salary INT
```

```
)
```

```
BEGIN
```

```
    INSERT INTO Emp VALUES (p_id, p_name, p_dept, p_salary);
```

```
END$$
```

```
DELIMITER ;
```

```
CALL insert_emp(8, 'Arun', 'HR', 38000);
```

```
SELECT * FROM Emp;
```

	emp_id	emp_name	dept	salary
▶	1	Asha	Sales	50000
	2	Ravi	HR	40000
	3	Meena	IT	60000
	4	John	IT	55000
	5	Priya	Sales	45000
	6	Kiran	HR	42000
	7	Sneha	IT	65000
	8	Arun	HR	38000
•	NULL	NULL	NULL	NULL

Procedure for Update:

```

26  DELIMITER $$
27  • CREATE PROCEDURE update_salary(
28      IN p_id      INT,
29      IN p_new_salary INT
30  )
31  BEGIN
32      UPDATE Emp
33      SET salary = p_new_salary
34      WHERE emp_id = p_id;
35  END$$
36  DELIMITER ;
37
38  • CALL update_salary(2, 45000);
39  • SELECT * FROM Emp;

```

Result Grid

	emp_id	emp_name	dept	salary
▶	1	Asha	Sales	50000
	2	Ravi	HR	45000
	3	Meena	IT	60000
	4	John	IT	55000
	5	Priya	Sales	45000
	6	Kiran	HR	42000
	7	Sneha	IT	65000
•	NULL	NULL	NULL	NULL

Procedure for Delete:

```

26 DELIMITER $$
27 • CREATE PROCEDURE delete_emp(
28     IN p_id INT
29 )
30 BEGIN
31     DELETE FROM Emp
32     WHERE emp_id = p_id;
33 END$$
34 DELIMITER ;
35
36 • CALL delete_emp(5);
37 • SELECT * FROM Emp;

```

Result Grid



Filter Rows:

	emp_id	emp_name	dept	salary
▶	1	Asha	Sales	50000
	2	Ravi	HR	40000
	3	Meena	IT	60000
	4	John	IT	55000
	6	Kiran	HR	42000
	7	Sneha	IT	65000
•	NULL	NULL	NULL	NULL

LAB – 10: -

Triggers –

```

AFTER UPDATE ON Emp
FOR EACH ROW
BEGIN
    INSERT INTO Emp_log (action, emp_id) VALUES ('UPDATE', NEW.emp_id);
END$$

CREATE TRIGGER before_emp_delete
BEFORE DELETE ON Emp
FOR EACH ROW
BEGIN
    IF OLD.salary > 60000 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Cannot delete employee with salary above 60000';
    END IF;
END$$

CREATE TRIGGER after_emp_delete
AFTER DELETE ON Emp
FOR EACH ROW
BEGIN
END$$

CREATE TRIGGER after_emp_insert
AFTER INSERT ON Emp
FOR EACH ROW
BEGIN
    INSERT INTO Emp_log (action, emp_id) VALUES ('INSERT', NEW.emp_id);
END$$

CREATE TRIGGER before_emp_update
BEFORE UPDATE ON Emp
FOR EACH ROW
BEGIN
    IF NEW.salary < OLD.salary THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary cannot be decreased';
    END IF;
END$$

CREATE TRIGGER after_emp_update

```

```
(6, 'Kiran', 'HR', 42000),  
(7, 'Sneha', 'IT', 65000);
```

```
DROP TRIGGER IF EXISTS before_emp_insert;  
DROP TRIGGER IF EXISTS after_emp_insert;  
DROP TRIGGER IF EXISTS before_emp_update;  
DROP TRIGGER IF EXISTS after_emp_update;  
DROP TRIGGER IF EXISTS before_emp_delete;  
DROP TRIGGER IF EXISTS after_emp_delete;
```

```
DELIMITER $$
```

```
CREATE TRIGGER before_emp_insert  
BEFORE INSERT ON Emp  
FOR EACH ROW  
BEGIN  
    IF NEW.salary <= 0 THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Salary must be greater than 0';  
    END IF;
```

```
CREATE TABLE Emp (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(30),  
    dept VARCHAR(30),  
    salary INT  
);
```

```
CREATE TABLE Emp_log (  
    log_id INT AUTO_INCREMENT PRIMARY KEY,  
    action VARCHAR(30),  
    emp_id INT,  
    log_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
INSERT INTO Emp VALUES
```

```
(1, 'Asha', 'Sales', 50000),  
(2, 'Ravi', 'HR', 40000),  
(3, 'Meena', 'IT', 60000),  
(4, 'John', 'IT', 55000),  
(5, 'Priya', 'Sales', 45000),
```

```
DELIMITER ;
```

```
INSERT INTO Emp VALUES (8, 'Arun', 'HR', 38000);
```

```
INSERT INTO Emp VALUES (9, 'Divya', 'IT', 48000);
```

```
UPDATE Emp SET salary = 45000 WHERE emp_id = 2;
```

```
UPDATE Emp SET salary = 52000 WHERE emp_id = 1;
```

```
DELETE FROM Emp WHERE emp_id = 6;
```

```
DELETE FROM Emp WHERE emp_id = 5;
```

```
SELECT * FROM Emp;
```

```
SELECT * FROM Emp_log;
```

	log_id	action	emp_id	log_time
►	1	INSERT	8	2026-10-08 20:28:46
	2	INSERT	9	2026-10-08 20:28:46
	3	UPDATE	2	2026-10-08 20:28:46
	4	UPDATE	1	2026-10-08 20:28:46
	5	DELETE	6	2026-10-08 20:28:46
	6	DELETE	5	2026-10-08 20:28:46
★	NULL	NULL	NULL	NULL